

Wio Tracker L1 Cross-Link Setup

Hardware checklist for bringing up the Meshtastic cross-link between beamrider-0003 and the second RPi node. Complete Phases 1–3 offline; Phase 4 and 5 require both nodes powered and in range.

Phase 1 — Initial connection and inspection

Both units ship pre-flashed with Meshtastic. Do this for each unit separately using a laptop before touching the RPis.

Easiest option — browser-based (no install required):

- ☐ Connect Wio Tracker to laptop via USB-C
- ☐ Open Chrome or Edge (WebSerial is not supported in Firefox/Safari)
- ☐ Go to client.meshtastic.org → Connect → Serial → select the device port
- ☐ Confirm the Meshtastic UI loads and shows firmware version and node ID (displayed as `!xxxxxxx` — record this for both units)
- ☐ Repeat for second unit

Alternative — Python CLI:

```
pip install meshtastic

# macOS — port appears as /dev/cu.usbmodem* (not ttyACM0)
meshtastic --port /dev/cu.usbmodem2101 --info

# Linux (RPi) — port appears as /dev/ttyACM0
meshtastic --port /dev/ttyACM0 --info
```

Look for `myInfo` → `myNodeNum` in the output. The node ID in `!xxxxxxx` format is the hex representation of that number.

macOS note: Always use `/dev/cu.usbmodem*` for outgoing serial connections on macOS. `tty.usbmodem*` exists but hangs indefinitely. Run `ls /dev/cu.usbmodem*` to find the exact port name after plugging in the device.

Phase 2 — Configure each Wio Tracker

Do for both units. The region and channel PSK must match for the two nodes to communicate. Easiest to do one unit at a time on the laptop via CLI.

macOS port: Use `/dev/cu.usbmodem*` (e.g. `/dev/cu.usbmodem2101`). Run `ls /dev/cu.usbmodem*` after plugging in to get the exact name. The commands below use `<PORT>` as a placeholder — substitute accordingly.

Install CLI if not already done:

```
pip install meshtastic
```

Step 2a — Set region (required before the radio will transmit):

```
meshtastic --port <PORT> --set lora.region US
```

The device will reboot. Wait ~5 s and reconnect for subsequent commands.

Step 2b — Set LoRa preset: `LONG_FAST` is appropriate for indoor bench use (SF11, ~1 kbps, good sensitivity). Matches the ≤ 1 kbps cross-link budget in the SBIR spec.

```
meshtastic --port <PORT> --set lora.modem_preset LONG_FAST
```

Step 2c — Configure a private channel on Unit A:

```
# Set channel name
meshtastic --port <PORT> --ch-index 0 --ch-set name satlab

# Generate a random PSK and apply it
meshtastic --port <PORT> --ch-index 0 --ch-set psk random
```

Step 2d — Export the config from Unit A and apply to Unit B:

```
# On Unit A – export full config to a YAML file
meshtastic --port <PORT> --export-config > wio-a.yaml

# Plug in Unit B, import the same config
meshtastic --port <PORT> --import-config wio-a.yaml
```

This is the most reliable way to guarantee both units share the identical PSK.

Step 2e — Set node names (do separately per unit after import):

```
# Unit A
meshtastic --port <PORT> --set-owner "beamrider-0003" --set-owner-short
"BR03"

# Unit B
meshtastic --port <PORT> --set-owner "beamrider-0004" --set-owner-short
"BR04"
```

Step 2f — Verify mesh connectivity before moving to the RPi:

- ☐ With both units powered (USB or battery), open `client.meshtastic.org` connected to Unit A
 - ☐ Confirm Unit B appears in the node list within ~60 s
 - ☐ Send a test message to Unit B via the UI and confirm receipt
 - ☐ Record both node IDs (`!xxxxxxx`) — needed for agent env vars
-

Phase 3 — RPi software setup (both nodes)

- ☐ Install the Meshtastic Python library: `pip install meshtastic --break-system-packages`
 - ☐ Plug the Wio Tracker into a USB port on the RPi
 - ☐ Identify both USB serial devices with both Arduino and Wio Tracker plugged in: `ls /dev/serial/by-id/` This lists symlinks with stable, hardware-keyed names — use these instead of `/dev/ttyACM*` (which can swap order on reboot)
 - ☐ Record the by-id path for the Arduino (will contain "Arduino" in the name) and for the Wio Tracker (will contain the nRF52840 USB descriptor)
 - ☐ Update the systemd service environment to use by-id paths:
`SATLAB_SERIAL_PORT=/dev/serial/by-id/usb-Arduino_LLC_Arduino_Uno_...-if00`
 - ☐ Add the new cross-link environment variable to the service:
`SATLAB_CROSSLINK_PORT=/dev/serial/by-id/usb-...<wio-tracker>...-if00`
 - ☐ Add the peer node ID from Phase 2: `SATLAB_PEER_NODE_ID=!xxxxxxx`
 - ☐ Verify the Meshtastic Python library can reach the Wio Tracker: `python import meshtastic.serial_interface iface = meshtastic.serial_interface.SerialInterface("/dev/serial/by-id/...") print(iface.myInfo) iface.close()`
 - ☐ Repeat on the second RPi node
-

Phase 4 — Cross-link connectivity test (both nodes powered, in range)

- ☐ From RPi-A, send a test message to RPi-B's node ID: `python import meshtastic.serial_interface iface = meshtastic.serial_interface.SerialInterface(CROSSLINK_PORT) iface.sendText("ping", destinationId=PEER_NODE_ID) iface.close()`
 - ☐ Confirm receipt on RPi-B via the receive callback or Meshtastic app
 - ☐ Measure round-trip latency — expect 1–5 s at LongFast preset indoors
 - ☐ Confirm bidirectional (send from RPi-B to RPi-A)
 - ☐ Stress test: send 10 messages in succession, confirm delivery rate
-

Phase 5 — Agent integration smoke test

- ☐ Start the satlab agent on beamrider-0003 with the updated env vars
- ☐ Confirm Arduino telemetry still ingests to Beamwarden (no regression)
- ☐ Confirm health_vector readings appear in Beamwarden

- ☐ With `crosslink.py` integrated (software work, done separately): - [] Confirm health vectors appear in the agent log as transmitted - [] Confirm peer health vectors are received and logged on the other node - [] Confirm Beamwarden shows health_vector readings from both node IDs

Environment variables added by this work

Variable	Description
<code>SATLAB_CROSSLINK_PORT</code>	Serial device for Wio Tracker (use by-id path)
<code>SATLAB_PEER_NODE_ID</code>	Meshtastic node ID of the peer RPi node (e.g. <code>!a1b2c3d4</code>)
<code>SATLAB_NODE_ID</code>	Stable UUID for this node's health vector identity

Notes

Packet size budget: Meshtastic maximum payload is 237 bytes. The health vector JSON payload (as currently structured) will exceed this at full verbosity. Before `crosslink.py` is written, measure the serialized size of `vector.to_payload()` and trim or compact if needed. The binary encoding specified in the proposal (≤ 256 bytes) is the Phase II target; for the Python prototype, compact JSON with short keys is a reasonable intermediate.

USB port ordering (RPi/Linux): Never rely on `/dev/ttyACM0` vs `/dev/ttyACM1` — the kernel assigns these on plug-in order. Always use `/dev/serial/by-id/` paths in env vars and the systemd service file.

macOS serial port naming: Use `/dev/cu.usbmodem*` for CLI commands, not `/dev/tty.usbmodem*`. The `tty.*` variant hangs indefinitely when meshtastic tries to open it. The CH340 driver (WCH `wch-ch34x-usb-serial-driver` cask) is not needed for the nRF52840-based Wio Tracker L1 — it enumerates as a native USB CDC device without additional drivers.

Battery operation: The 3000 mAh batteries allow the Wio Trackers to operate untethered. For bench testing, USB power from the RPi is simpler. Battery operation becomes relevant when testing cross-link range or simulating ground contact blackouts.